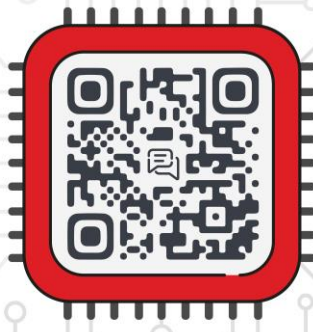




TD

5



5



7.5 sp

كلية الهندسة المعلوماتية

السنة الأولى

العلاقات بين الكلاسات في جافا.

د. ماهر صارم



البرمجة 2

RB Informatcs; 27/4/2026

محتوى مجاني غير مخصص للبيع التجاري

علاقة الارتباط (Association)

هي علاقة تربط بين كلاسَيْن، تسمح هذه العلاقة لأي كلاس بأن يتعرف على كلاس آخر ويستخدم الخدمات التي يقدمها، مع بقاء كل كلاس منهما مستقلاً تماماً عن الآخر.

لها نوعان:

1- التجميع (Aggregation):

يمتلك الكلاس الأساسي كائنات (objects) من كلاس آخر، لكن هذه الكائنات الفرعية تظل مستقلة بحيث إذا حذف الكلاس الأساسي تظل الكائنات الفرعية موجودة ولا تتأثر.

مثال من الواقع لتوضيح الفكرة:

العلاقة بين المدرسة والمدرسين (حيث المدرسة هي class والمدرسين هم الـ objects):
المدرسة تضم مجموعة من الأساتذة، لكن إذا أغلقت المدرسة (حذف الكلاس الأساسي) لن يختفوا الأساتذة بل سيبقون موجودين ويمكنهم الانتقال لمدرسة أخرى.

الخلاصة: الأستاذ يمكنه الوجود والعمل حتى بلا وجود هذه المدرسة المحددة.

مثال بالكود:

شرح الكود:

في الكود التالي لدينا:

1- كلاس الطالب Student وكلاس قسم (فرع) وهو Department.

```
1- public class Student {
2-     private int id;
3-     private String name;
4-     Student(int id,String name){
5-         this.id=id;
6-         this.name=name;
7-     }
8- }
```

```

1- import java.util.List;
2- public class Department {
3-     private String name;
4-     private List<Student> students;
5-     Department(String name, List students){
6-         this.name=name;
7-         this.students=students;
8-     }
9- }

```

علاقة التجميع (Aggregation) هي بين الكلاس Department والكلاس Student حيث أن كل كائن (object) في Department يحوي بداخله مجموعة كائنات Student في (List) لكن هذه الكائنات مستقلة بذاتها، أي في حالة لم يكن كائن Department موجود ستبقى كائنات Student موجودة بشكل طبيعي.

```

1- import java.util.ArrayList;
2- import java.util.List;
3- public class Main {
4-     public static void main(String[] args){
5-         Student S1=new Student(111,"Majd");
6-         Student S2=new Student(222,"Ahmad");
7-         Student S3=new Student(333,"Lulu");
8-         Student S4=new
Student(444,"AboShoky");
9- List<Student>students=new ArrayList<>();
10-     students.add(S1);
11-     students.add(S2);
12-     students.add(S3);
13-     students.add(S4);
14- Department D=new
Department("ITE",students);
15- }

```

الخلاصة: القسم يحتاج لطلاب لكي يعمل، ولكن الطلاب لا تتأثر (ستبقى موجودة) إذا اختفى هذا القسم.

آلية عمل الـ Aggregation هنا:

- الأسطر من 5 إلى 8 تم إنشاء طلاب مستقلين في الذاكرة بأسماء وأرقام مختلفة، حيث يتم وضعها في قائمة فارغة من النوع ArrayList كما في السطر 8.

- أنشأنا List من نوع Student وأضفنا عليها كائنات Student التي قمنا بإنشائها (الأسطر من 10 إلى 13) باستخدام الدالة add().

- الخطوة الثالثة: في السطر 14 نقوم بإنشاء كائن Department ونعطيه

الاسم ITE، ونمرر له قائمة الطلاب التي ملأناها للتو، وهنا تحدث عملية الربط بشكل فعلي، حيث نقوم بوضع List الكائنات Student التي أنشأناها ضمن كائن Department المنشأ.

ملاحظة:

تعريف الـ List:

الـ List هو واجهة (Interface) وليس كلاس، ويعتبر أعم وأشمل من الـ ArrayList، لكن لا يمكن إنشاء كائن مباشرة عبره بما أنه واجهة، لذلك نستخدم باني Constructor لكلاس ملموس يطبق هذه الواجهة، ولهذا استعملنا باني الـ ArrayList.



لماذا عرفنا متغير Student من النوع List؟

لسهولة تعديل الكود، لأنه إذا أردت نوع القائمة إلى نوع آخر، لن تضطر لإعادة كتابة الكود أو إضافة جميع العناصر إلى List من جديد.

2- التكوين (Composition):

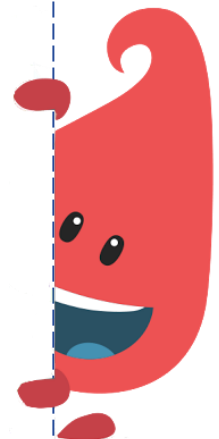
هي أقوى أنواع الارتباط، هنا الكائن الفرعي لا يمكنه العيش دون الكلاس الأساسي، فهو جزئ لا يتجزأ منه.

مثال من الواقع للتوضيح:

العلاقة بين الكتاب (class) والأوراق (objects) هي علاقة تكوين (composition).
إذا قمت بإتلاف الكتاب فإن الأوراق التابعة له ستزول معه حتماً.
لا يمكن لهذه الأوراق أن تؤدي وظيفتها وتوجد بشكل مستقل دون الكتاب.
أي أن حياة الأوراق مرتبطة تماماً بحياة الكتاب، فمع زوال الكل (الكتاب) يزول الجزء (الأوراق).

مثال بالكود:

```
public class Job {  
    private String role;  
    private double salary;  
    Job(String role, double salary){  
        this.role=role;  
        this.salary=salary;  
    }  
    public void setSalary(double salary) {  
        this.salary = salary;  
    }  
    public double getSalary(){  
        return salary;  
    }  
}
```



```
public class Employee {  
    private String name;  
    private Job job;  
    Employee(String name,String role,double salary){  
        this.name=name;  
        job =new Job(role,salary);  
    }  
    public void setEmployeeSalary(double salary){  
        job.setSalary(salary);  
    }  
    public double getEmployeeSalary(){  
        return job.getSalary();  
    }  
}
```

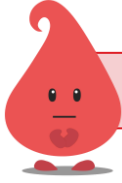
```
1- public class Main {  
2-     public static void main(String[] args) {  
3-         Employee e1=new  
           Employee("Majd","Manager",1500);  
4-         System.out.println(e1.getEmployeeSalary());  
5-         e1.setEmployeeSalary(2000);  
6-         System.out.println(e1.getEmployeeSalary());  
           }  
       }
```

شرح الكود:

من الكود السابق لدينا كلاس موظف Employee وكلاس وظيفة Job.

■ نوع العلاقة:

العلاقة بين الكلاس Employee و Job هي علاقة تكوين (composition) حيث أن كائن Employee يحتوي بداخله على كائن من نوع Job.



لماذا العلاقة هي علاقة تكوين (composition)؟

لأننا قمنا بإنشاء كائن الوظيفة (newJob) داخل المنشئ أو الباني (constructor) الخاص بالموظف. هذا يعني أن كائن الوظيفة مرتبط وجودياً بالموظف، فإذا حذف الموظف تلاشت وظيفته معه، ولا يمكن للوظيفة أن توجد في هذا النظام بدون موظف يشغلها.

■ آلية الوصول للراتب (getSalary):

في كلاس Job توجد دالة getSalary() تعيد قيمة الـ Salary.

في كلاس Employee أنشأنا دالة اسمها getEmployeeSalary() عند استدعاؤها من أجل e1 (e1.getEmployeeSalary()) كما في السطر 4 في الـ main يقوم كائن الموظف بالذهاب لكائن الوظيفة الخاص به باستعمال getSalary() المعرفة ضمنه وعن طريقها يتم إعادة قيمة الـ Salary.

■ كيفية تعديل قيمة الراتب (setSalary):

في كلاس Job توجد دالة setSalary (double salary) لتحديث قيمة salary.

في كلاس Employee أنشأنا دالة setEmployeeSalary (double salary) التي يستعملها الموظف ليأخذ قيمة.

في مثالنا في السطر 5 ضمن الـ Main مررنا قيمة (2000) وبعدها يمررها لكائن الموظف عند استدعاء تابع (double setEmployeeSalary (salary).

ويعرر داخله قيمة 2000 والذي يقوم باستعمال تعليمة Job.setSalary(2000) وعندها يتم داخل كلاس الوظيفة تعديل قيمة الـ Salary.

إضافة أريسيزية:

الهدف الأساسي من أسطر الـ main (الأسطر 4<=6) هو إظهار كيف يتحكم كائن الموظف الخارجي بكائن الوظيفة الداخلي المخفي عنه (عن طريق الوسيط المتوفر في الدوال)، سواء عند القراءة أو عند التعديل.

انتهت المحاضرة الخامسة

Good luck!